

Práctica del Laboratorio 4

Arquitectura del Computador

5 de Agosto de 2026

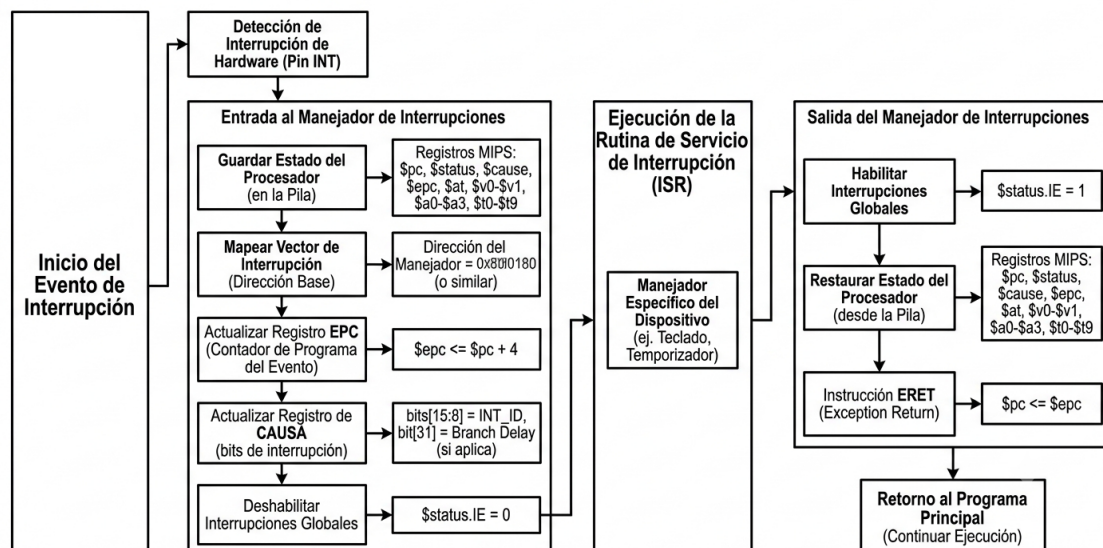
1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende):

Ocurre el evento → el hardware activa la línea INT, el procesador guarda el estado (EPC, CAUSA), deshabilita interrupciones y salta al manejador.

Se ejecuta la ISR → identifica el dispositivo (lectura de INT_ID), atiende la interrupción y realiza el trabajo necesario.

Se retorna → se restauran registros, se habilita IE, se ejecuta ERET y el PC vuelve a EPC, continuando el programa principal.

DIAGRAMA DEL CICLO DE UNA INTERRUPCIÓN DE HARDWARE EN MIPS32



2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?:

El sondeo (polling) consiste en que el procesador consulta constantemente el estado de los dispositivos de E/S en un bucle, lo que desperdicia ciclos de CPU aunque no haya datos pendientes, pero su implementación es simple y predecible.

En cambio, las interrupciones permiten que el dispositivo "avise" al procesador solo cuando necesita atención, liberando la CPU para otras tareas y mejorando la eficiencia global del sistema. Sin embargo, las interrupciones introducen una sobrecarga adicional por el cambio de contexto (guardar/restaurar registros, ejecutar la ISR y retornar con ERET), mientras que el sondeo no tiene ese costo de latencia.

El sondeo es adecuado para sistemas pequeños o con pocos dispositivos, mientras que las interrupciones son indispensables en sistemas multitarea o con múltiples periféricos que requieren respuesta rápida y eficiente.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?:

Con las interrupciones, el procesador no tiene que estar preguntando todo el tiempo si un dispositivo necesita atención, así que puede dedicarse a otras tareas mientras no pasa nada. Cuando el dispositivo realmente necesita algo, él mismo avisa y la CPU lo atiende en ese momento, sin perder el tiempo revisando estados vacíos. Esto hace que el procesador trabaje de forma mucho más inteligente, porque solo gasta ciclos cuando hay trabajo real que hacer. Al final, el sistema responde más rápido, consume menos energía y puede manejar varios periféricos a la vez sin que el rendimiento se desplome.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?:

Registro	Función
\$status	Controla la habilitación global de interrupciones y la máscara de periféricos.
\$cause	Indica qué interrupción ocurrió y si estaba en un delay slot.
\$epc	Guarda la dirección de retorno para volver al programa principal tras la interrupción.
\$badvaddr	Almacena direcciones inválidas (solo para excepciones de memoria).

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?:

Es necesario guardar el contexto porque la rutina de interrupción va a usar los registros del procesador para hacer su trabajo, y si los modifica sin guardarlos, al volver al programa

principal este encontrará sus datos corruptos y fallará. El programa principal estaba ejecutándose con unos valores concretos en registros como \$t0-\$t9, \$a0-\$a3 o \$v0-\$v1, y no sabe que ocurrió una interrupción, por lo que espera encontrar esos valores intactos.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32:

a) Situaciones que generan excepciones en MIPS32:

- Desbordamiento aritmético (ADD/SUB con resultado fuera de rango).
- Fallo de dirección (acceso a memoria no alineada o fuera de rango).
- Instrucción no válida (código de operación no reconocido).
- Llamada al sistema (SYSCALL) o instrucción de trap (BREAK).

b) Etapas del pipeline que provocan excepción:

- **Decodificación (ID):** detecta instrucciones no válidas, SYSCALL o BREAK.
- **Ejecución (EX):** detecta desbordamiento aritmético.
- **Memoria (MEM):** detecta fallos de alineación o página no presente (TLB miss).
- **Escritura (WB):** generalmente no genera excepciones, salvo casos muy específicos de coprocesador.

7. Estrategias de tratamiento de excepciones e interrupciones:

a) Diferencias entre interrupciones y excepciones:

Interrupciones	Excepciones
Asíncronas: ocurren en cualquier momento, por eventos externos al procesador (hardware).	Síncronas: ocurren en un punto fijo de la ejecución, provocadas por la propia instrucción que se está ejecutando (desbordamiento, instrucción inválida, etc.).

b) Estrategias para tratar excepciones en MIPS32: Estrategia 1 - Manejador único:

- Todas las excepciones saltan a una dirección fija (0x80000180 para excepciones, 0x80000000 para reset).

- El manejador lee el registro `$cause` para determinar el tipo de excepción y bifurca a la rutina específica.
- Es simple y ocupa poco espacio, pero requiere lógica de decodificación adicional.

Estrategia 2 - Manejador con tabla de vectores:

- Cada tipo de excepción tiene su propia dirección de entrada (según el `ExcCode` o `IP`).
- El hardware calcula la dirección base + desplazamiento, saltando directamente a la ISR específica.
- Es más rápido porque evita la decodificación, pero consume más memoria de código.

¿Cómo se redirige la ejecución hacia la rutina de servicio?: El procesador:

1. Guarda la dirección de retorno en `$epc` (PC de la instrucción que falló o la siguiente, según el bit `BD` de `$cause`).
2. Guarda la causa en `$cause`.
3. Deshabilita interrupciones (`IE = 0` en `$status`).
4. Carga el PC con la dirección del manejador (fija o vectorizada).

Función del registro EPC: Almacena la dirección de retorno para que, tras ejecutar `ERET`, el procesador pueda volver al punto exacto donde fue interrumpido o donde ocurrió la excepción, continuando la ejecución normal o reintentando la instrucción fallida (si es recuperable).

8. Habilitación de interrupciones en dispositivos y procesador:

a) Habilitación de interrupciones: Teclado:

Se escribe un 1 en el bit de interrupt enable de su registro de control (ej. bit 0 del registro `STATUS` del periférico). Así el teclado genera una interrupción al pulsar una tecla o al llenar su buffer.

Pantalla:

Se activa el bit de habilitación de interrupciones en su registro de control (ej. bit `IE` del registro `DISPLAY_CTRL`). Normalmente se configura para que interrumpa cuando termine una transferencia DMA o cuando el buffer de salida esté vacío.

Procesador (registro \$status):

- Bit 0 (`IE`) → se pone a 1 para habilitar interrupciones globales.
- Bits 8–15 (`IM[7:2]`) → máscara de interrupciones: se ponen a 1 los bits correspondientes a los periféricos que queremos atender (ej. bit 8 para teclado, bit 9 para pantalla, etc.).

b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos pero no en el procesador?: Sería como tener el timbre de casa funcionando, pero con los oídos tapados. Los dispositivos generarían las interrupciones, pero el procesador las ignoraría por completo porque el bit IE del `$status` está a 0. El sistema nunca atendería los eventos, los periféricos se quedarían bloqueados esperando servicio y el programa principal seguiría ejecutándose sin enterarse de nada, provocando pérdida de datos o un funcionamiento incorrecto.

9. Procesamiento de interrupciones:

a) Paso a paso de interrupción de reloj: Cuando el temporizador alcanza el valor de comparación, el hardware guarda automáticamente el PC en `$epc`, registra la causa en `$cause`, deshabilita interrupciones (`IE=0`) y salta al manejador. El software entonces guarda en la pila los registros temporales (`$t0-$t9`, `$a0-$a3`, `$v0-$v1`, `$at`) y también copias de `$epc` y `$status`. Luego lee `$cause` para confirmar que es interrupción de reloj, ejecuta la ISR (actualiza ticks, programa el próximo `$compare` y revisa tareas pendientes), restaura todos los registros desde la pila y ejecuta `ERET`, que recupera el PC desde `$epc` y reactiva las interrupciones globales.

b) Importancia de guardar el contexto: Es vital porque el programa principal no sabe que ocurrió una interrupción y espera que sus registros sigan intactos. Si la ISR los modifica sin restaurarlos, al volver el programa encontrará datos corruptos y fallará. Guardar el contexto es como hacer una "foto" del estado del procesador para devolverlo exactamente igual, asegurando que la interrupción sea completamente transparente para el código interrumpido.

10. Interrupciones de reloj y control de ejecución:

a) Uso de la interrupción de reloj para control de ejecución: La interrupción de reloj permite al sistema operativo tomar el control periódicamente y comprobar cuánto tiempo lleva ejecutándose un programa. Si el programa supera su tiempo máximo sin ceder la CPU, el SO asume que está en un bucle infinito y lo finaliza forzosamente, liberando los recursos y evitando que monopolice el procesador, garantizando así la estabilidad del sistema.

b) Qué hacer si el programa finaliza antes de la interrupción: Si el programa termina voluntariamente antes de que llegue la interrupción, el SO debe cancelar el contador de tiempo asociado a ese proceso y reorganizar la planificación. Así, cuando llegue la siguiente interrupción de reloj, no se intentará finalizar un programa que ya no existe, y el temporizador se reprogramará correctamente para el siguiente proceso en ejecución.

11. Análisis y Discusión de los Resultados:

Los dos códigos implementan sistemas en ensamblador MIPS utilizando memory-mapped I/O para la entrada de teclado y la syscall 30 para la temporización mediante sondeo (polling), evitando el uso de interrupciones de hardware.

Captura de caracteres:

- Ejecuta un intervalo fijo de 20 segundos capturando únicamente letras mayúsculas (A-Z, ASCII 65-90).
- Almacena los datos en un búfer circular de 100 caracteres y muestra el resultado al cumplirse el tiempo.
- Su estructura es un bucle principal lineal y continuo.

Semáforo secuencial:

- Inicia en verde y espera la pulsación de la tecla 's' (ASCII 115) para iniciar una secuencia cíclica: 20s en verde, 10s en amarillo y 30s en rojo.
- Utiliza una función temporizadora modular reutilizable (jr \$ra) que mide milisegundos mediante la syscall 30.
- Su estructura corresponde a una máquina de estados finitos.

Aspecto	Caracteres	Semáforo
Método de E/S	Sondeo (Memory-mapped I/O)	Sondeo (Memory-mapped I/O)
Temporización	Bucle activo con syscall 30	Función reutilizable con syscall 30
Filtrado de entrada	Solo letras A-Z	Únicamente tecla 's'
Estructura	Bucle principal con reinicio automático	Máquina de estados con secuencia fija
Modularidad	Código lineal	Función temporizadora separada

Conclusión

Ambos enfoques priorizan el control total de los tiempos y la simplicidad del código por encima de la eficiencia en el uso de la CPU, adaptándose el primero a la adquisición de datos por tiempo y el segundo al control secuencial de estados.